

PlatePlanner

AI-Powered Meal Planning & Recipe Recommendation System

Technical Project Report

San Jose State University · CMPE 295B Master's Project

Sandilya Chimalamarri · Sai Priyanka Bonkuri · Pavan Charith Devarapalli · Sai Dheeraj Gollu

April 2026

1. Executive Summary

PlatePlanner is an AI-powered, graph-enhanced meal planning and recipe recommendation platform developed as a San Jose State University CMPE 295B capstone project. The system addresses a common problem: users have a set of available ingredients and dietary constraints, but lack a fast, intelligent way to discover suitable recipes, plan meals, generate shopping lists, and navigate ingredient substitutions.

The platform combines a FastAPI backend, a Neo4j property-graph database, PostgreSQL relational storage, Redis caching, and a FAISS vector-search index over 2.2 million recipes. A hybrid semantic + ingredient-overlap ranking algorithm improves recall by 23% over a pure vector-search baseline. A React Native mobile client exposes the full feature set on iOS and Android.

2.2M+ Total Recipes	95%+ Test Coverage	<20ms FAISS Latency	+23% Recall Boost	100+ RPS Throughput
---------------------------------------	--------------------------------------	---------------------------------------	-------------------------------------	---------------------------------------

Table 1 — Key system performance metrics

2. Core Architecture

PlatePlanner follows a **4-Tier architecture** that cleanly separates concerns across API, service, data-persistence, and ML-inference layers.

2.1 Tier 1 — API Layer (FastAPI)

The entry point is `src/api/app.py`. FastAPI 0.115 runs on Uvicorn (ASGI, fully async). Eight REST routers handle: *auth*, *users*, *meal_plans*, *shopping_lists*, *nutrition*, *recommendations*, *user_meals*, and *pantry*. Request/response validation is handled by Pydantic v2. CORS origins are configurable via the `ALLOWED_ORIGINS` environment variable.

2.2 Tier 2 — Service Layer (Domain-Driven Design)

All business logic lives in `src/services/`. Key services:

Service File	Responsibility
<code>neo4j_service.py</code>	Graph database wrapper; Cypher query execution
<code>retrieval_service.py</code>	FAISS vector search + hybrid re-ranking
<code>substitution_service.py</code>	Pantry-aware ingredient substitution
<code>meal_plan_service.py</code>	AI-driven weekly meal plan generation
<code>shopping_list_service.py</code>	Multi-recipe aggregation & unit normalisation (730+ lines)
<code>nutrition_service.py</code>	USDA-backed macronutrient analysis & goal tracking
<code>rag_service.py</code>	LLM-powered recipe generation / adaptation (RAG pipeline)
<code>dietary_classifier.py</code>	Rule-based dietary tag & allergen detection
<code>external_recipe_service.py</code>	Spoonacular API fallback integration

Table 2 — Service layer modules

2.3 Tier 3 — Polyglot Persistence Layer

Each storage technology is chosen to match its data’s access pattern:

Database	Role
PostgreSQL 15	Users, meal plans, shopping lists, nutrition logs — ACID, relational
Neo4j 5.x	Recipe–ingredient graph, substitution edges, similarity edges
Redis 7	Session tokens, API response cache, rate-limiting, job queue
SQLite 3	Recipe metadata cache (2.2M rows); optimised for fast lookup

Table 3 — Polyglot persistence layer

2.4 Tier 4 — ML / AI Inference Layer

CPU-bound inference tasks run in a dedicated thread pool to avoid blocking the async event loop. The inference pipeline is:

- **1. Query embedding** — User ingredients text → 384-d vector via SentenceTransformer.
- **2. ANN search** — FAISS IVF-PQ index returns top-500 nearest recipe vectors (<20ms).
- **3. Hybrid re-ranking** — Combines cosine similarity (40%) with discrete ingredient overlap (60%) to produce a final combined score.
- **4. Dietary filtering** — Rule-based classifier removes recipes that violate user restrictions or contain allergens.
- **5. LLM fallback** — When FAISS matches are sparse, the RAG service calls OpenAI / Gemini / Ollama to synthesise novel recipes.

3. Machine Learning Models

3.1 Sentence-Transformer Embedding Model

The primary embedding model is **all-MiniLM-L6-v2** from the sentence-transformers library (v4.1.0). It produces 384-dimensional dense vectors that capture semantic meaning of recipe titles and ingredient lists. The model is lightweight enough to run on CPU with sub-50ms encoding latency for typical queries. An optional **finetuned-recipe-encoder** (domain-adapted on food/recipe text) is auto-loaded when present at `src/data/models/recipe_suggestion/finetuned-recipe-encoder/`.

3.2 FAISS Vector Index

All 2.2 million recipe embeddings are indexed with **FAISS 1.11.0**. Two variants are maintained:

File	Index Type	Size	p95 Latency	Usage
recipe_index_opt.faiss	IVF-PQ (quantised)	~90 MB	<20ms	Production
recipe_index.faiss	Flat (exact)	~12 GB	~200ms	Evaluation only

Table 4 — FAISS index variants. IVF-PQ (Inverted File + Product Quantisation) reduces RAM from ~12 GB to <200 MB while maintaining sub-20ms recall.

3.3 Hybrid Ranking Algorithm

Raw FAISS cosine similarity alone struggles to surface recipes that share many of the user's exact ingredients but differ in cooking style or phrasing. The hybrid ranker combines two complementary signals:

combined_score = 0.4 × cosine_similarity + 0.6 × normalised_ingredient_overlap

The 60/40 weighting was determined empirically and yields a **+23% recall improvement** over pure vector search on an internal evaluation set.

3.4 Named Entity Recognition (SpaCy)

SpaCy en_core_web_sm (v3.8.5) is used during data-ingestion pipelines to extract ingredient entities from unstructured recipe text. It achieves 87% precision and 82% recall on 1.5M unstructured ingredient strings. Parsed entities feed the Neo4j graph and FAISS metadata.

3.5 Dietary Classification

A rule-based DietaryClassifier (no external model required) flags recipes with dietary and allergen tags. It handles tricky compound ingredients (e.g., *coconut milk* is dairy-free; *soy sauce* contains gluten and soy). Supported tags:

- Dietary: vegan, vegetarian, gluten-free, dairy-free
- Allergens: peanuts, tree nuts, soy, fish, shellfish, eggs, dairy, gluten

3.6 LLM Backends (RAG Pipeline)

When FAISS retrieval returns insufficient results the RAG service generates novel recipes via a configurable LLM backend. The auto-detection chain is:

Provider	Model	Activation	Trade-off
OpenAI	gpt-4o-mini (default) or gpt-4	OPENAI_API_KEY set	Best quality
Google Gemini	gemini-2.0-flash (with fallback chain)	GOOGLE_API_KEY set	Fast, free tier

Provider	Model	Activation	Trade-off
Ollama (local)	llama3 or configurable model	Default fallback	Privacy, no cost

Table 5 — LLM backend options

4. Datasets & Data Sources

4.1 RecipeNLG (Primary Dataset)

RecipeNLG is the backbone of the recommendation engine. It contains **2,231,150 recipes** scraped from cookbooks.com and other sources, released as a benchmark for natural language generation in the cooking domain. Each record includes:

- **title** — Recipe name (used for fuzzy deduplication and display)
- **ingredients** — JSON list of raw ingredient strings
- **directions** — JSON list of cooking step strings
- **link** — Source URL
- **source** — Origin domain
- **NER** — SpaCy-parsed named entities (ingredient tokens only)

The raw CSV is stored at `src/data/raw/RecipeNLG_dataset.csv` (approx. 3–4 GB). A cleaned and indexed SQLite copy (~90 MB) is maintained at `src/data/models/recipe_suggestion/recipes.db` for fast lookup.

4.2 International Recipe Datasets

Three supplementary datasets extend coverage beyond English-language recipes:

File	Source	Value Added
food_com_recipes.csv	Food.com corpus	Community-submitted, diverse cuisines
recipes_with_nutrition.csv	Nutrition-enriched recipes	Adds calorie/macro data per recipe
cooking_recipes_large.csv	Large cooking collection	Extended cuisine diversity

Table 6 — International recipe datasets (`src/data/raw/international/`)

4.3 USDA Nutrient Data

Macronutrient values (calories, protein, carbohydrates, fat) per ingredient are sourced from the **USDA FoodData Central** database and stored in the `IngredientNutrition` PostgreSQL table. This powers the nutrition analysis and goal-tracking features.

4.4 Spoonacular API

When the local recipe corpus does not contain a suitable match for a highly specific query, the `ExternalRecipeService` calls the **Spoonacular REST API** as a 3rd-tier fallback (after FAISS and before LLM generation). Results are normalised and returned in the same schema as local results.

4.5 Processed Data (ETL Outputs)

File (<code>src/data/processed/</code>)	Contents
recipes.csv	Deduplicated recipe metadata
ingredients.csv	Unique ingredient vocabulary
recipe_ingredients.csv	Recipe–ingredient join table (for graph bootstrap)
recipe_metadata.csv	Extended attributes: cuisine, cook time, estimated calories

File (src/data/processed/)	Contents
substitution_edges_with_context_cleaned.csv	Neo4j substitution relationship edges with context labels

Table 7 — Processed data artefacts used for graph and index bootstrap

5. Ingredient Substitution System

Ingredient substitution is one of PlatePlanner's differentiating features. It combines a **Neo4j property graph** with **pantry-awareness** to suggest contextually appropriate swaps.

5.1 Neo4j Graph Structure

The graph has three primary node types and three relationship types:

Name	Type	Description
Recipe	Node	Recipe entity with title, tags, cuisine
Ingredient	Node	Normalised ingredient entity
HAS_INGREDIENT	Edge (Recipe→Ingredient)	Recipe contains ingredient (with quantity)
SUBSTITUTES_WITH	Edge (Ingredient→Ingredient)	Substitution with <i>context</i> property
SIMILAR_TO	Edge (Recipe→Recipe)	Semantic recipe similarity for discovery

Table 8 — Neo4j graph schema

5.2 Context-Aware Substitution Edges

The SUBSTITUTES_WITH relationship carries a **context** property that distinguishes *why* an ingredient is being substituted. This prevents semantically valid but culinarily wrong swaps, for example:

Substitution Pair	Context
eggs → flax egg	baking (binding)
eggs → tofu scramble	primary protein (scramble)
cornstarch → arrowroot	thickening
butter → coconut oil	fat/moisture (baking)
milk → oat milk	dairy-free liquid substitute

Table 9 — Example context-aware substitution edges

5.3 Hybrid Substitution Algorithm

For any query ingredient the service executes a two-stage lookup:

- **Stage 1 — Direct edges:** Traverse SUBSTITUTES_WITH edges in Neo4j. These are curated or similarity-derived relationships with explicit context labels. Query latency <10ms (p95) even for densely connected ingredient nodes.
- **Stage 2 — Co-occurrence analysis:** Identify ingredients that frequently appear together in the same substitution context across the 2.2M recipe corpus.
- **Merge:** Results are combined with alpha-weighting (default $\alpha = 0.9$), heavily favouring direct graph edges over statistical co-occurrence.

User satisfaction with dietary substitution recommendations measured at **92%** in internal evaluations.

5.4 Pantry-Aware Substitution

When a user requests substitutions for an entire recipe, SubstitutionService compares recipe ingredients against the user's pantry inventory (UserPantryItem table) and returns three lists:

- **Have:** Ingredients the user already owns (no substitution needed).
- **Missing:** Ingredients not in pantry — candidate for purchase or substitution.
- **Substitutable:** Missing ingredients for which a pantry item is a valid substitute, ranked by: 1) direct Neo4j edge, 2) similarity score.

5.5 Unit Conversion & Shopping List Consolidation

A key enabling component is the unit-conversion engine (src/utils/unit_converter.py, 9,000+ lines). It:

- Normalises heterogeneous units: *1 cup* → *240 ml*, *1 lb* → *454 g*, *1 tsp* → *5 ml*.
- Applies volumetric / mass conversions with density heuristics per ingredient category.
- Uses **fuzzy Levenshtein matching** to consolidate ingredient name variants (e.g., *"diced tomatoes"* and *"whole tomatoes"* → *tomatoes*).
- Achieves **95%+ accuracy** on complex multi-recipe shopping list aggregations.

6. PostgreSQL Data Schema

All user, meal-plan, nutrition, and transactional data is stored in PostgreSQL 15 via SQLAlchemy ORM. Alembic manages schema migrations.

Model	Key Fields
User	id (UUID), email, hashed_password, full_name, is_premium, is_admin, auth_provider
UserPreferences	dietary_restrictions[], allergies[], cuisine_preferences[], calorie/macro targets, max_cooking_time, budget
MealPlan	week_start/end_date, status, total macros, is_valid, validation_issues (JSON)
MealPlanItem	recipe_id (Neo4j ref), recipe_title, servings, macros, prep_time, estimated_cost
ShoppingList	status (active/completed/archived), total_estimated_cost, item counts
ShoppingListItem	ingredient_name, normalized_name, quantity, unit, category, estimated_price, store_prices (JSON)
NutritionGoal	daily calorie/protein/carb/fat targets per user
NutritionLog	Per-day actual macro tracking
UserMeal	Custom meals created by user
UserPantryItem	ingredient, quantity, unit, expiry_date, purchase_date
IngredientNutrition	USDA nutrition facts per ingredient (calories, protein, carbs, fat per 100g)

Table 10 — PostgreSQL ORM models

7. Key API Endpoints

Endpoint	Description
POST /suggest_recipes	3-tier federated search: FAISS → Spoonacular → LLM; accepts ingredients, dietary filters, allergies, cuisines
GET /recipes/{title}	Full recipe detail; fallback chain: SQLite → Neo4j → CSV
GET /substitute	Get substitutes for one ingredient with optional context and hybrid flag
POST /recipes/{title}/substitutions	Full recipe substitution check against user pantry
POST /meal-plans/generate	AI-driven weekly meal plan generation with constraint satisfaction
POST /shopping-lists/generate	Generate aggregated shopping list from a meal plan
GET /nutrition/analysis	Macronutrient breakdown for a given date range
POST /nutrition/log	Log a meal and update nutrition totals
GET /nutrition/insights	Health trends and goal-progress summary
POST /auth/register	JWT-based user registration
POST /auth/login	JWT token issuance (+ Google OAuth2 support)

Table 11 — Core REST API endpoints

8. Technology Stack

Library / Tool	Layer	Purpose
FastAPI 0.115	Backend	Async REST framework (Uvicorn ASGI)
Pydantic v2.11	Backend	Request/response validation & serialisation
SQLAlchemy 2.0	ORM	PostgreSQL ORM + Alembic migrations
sentence-transformers 4.1	ML	all-MiniLM-L6-v2 recipe embedding model
FAISS 1.11	ML	IVF-PQ vector index over 2.2M recipes
SpaCy 3.8	NLP	Named entity recognition for ingredient parsing
scikit-learn 1.6	ML	Evaluation utilities, distance metrics
neo4j-driver 5.28	DB	Neo4j Bolt protocol client
Redis 7	Cache	Session cache, rate limiting, job queue
PostgreSQL 15	DB	Primary relational store
SQLite 3	DB	Local recipe metadata cache
OpenAI SDK	LLM	GPT-4o-mini / GPT-4 recipe generation
google-genai	LLM	Gemini 2.0 Flash recipe generation
Ollama	LLM	Local model inference (llama3)
thefuzz / python-Levenshtein	Utils	Fuzzy string matching for deduplication
pint 0.23	Utils	Unit conversion
React Native + Expo	Frontend	Cross-platform iOS/Android mobile client
Docker + Docker Compose	Infra	Container orchestration

Table 12 — Full technology stack

9. Performance & Evaluation

Metric	Value	Notes
FAISS query latency (500-NN)	<20ms p95	IVF-PQ quantisation
Neo4j substitution traversal	<10ms p95	Dense ingredient node clusters
End-to-end HTTP response	<80ms p95	Async FastAPI + connection pooling
Throughput	100+ RPS	Locust load tests
SpaCy NER precision	87%	1.5M unstructured ingredient strings
SpaCy NER recall	82%	Same evaluation set

Metric	Value	Notes
Shopping list accuracy	95%+	Multi-recipe aggregation benchmark
Hybrid ranking recall gain	+23%	vs. pure FAISS cosine similarity
Substitution user satisfaction	92%	Dietary substitution user study
FAISS index size (IVF-PQ)	~90 MB	Down from ~12 GB flat index
Data ingestion (10K recipes → FAISS)	~5 min	On standard CPU hardware

Table 13 — Measured performance benchmarks

10. Deployment & Infrastructure

The full stack is containerised with **Docker** and orchestrated via **Docker Compose**. A single docker-compose.yml declares five services: FastAPI, PostgreSQL, Neo4j, Redis, and a data-init bootstrap container.

- **Local dev:** .env file; all services on localhost.
- **Production:** .env.prod with TLS; PostgreSQL hosted on **Neon** (serverless Postgres); Neo4j on **AuraDB**; FastAPI on **Railway**.
- **Mobile:** React Native Expo build targeting iOS and Android.
- **Admin:** SQLAdmin dashboard mounted at /admin for content management.

11. Team Contributions

Name	Role	Key Contributions
Sandilya Chimalamarri	ML Research Lead	Sentence-transformer + FAISS pipeline (10–20ms); hybrid ranking algorithm (+23% r
Sai Priyanka Bonkuri	Graph DB Architect	Neo4j schema design; context-aware substitution graph; traversal optimisation (3× sp
Pavan Charith Devarapalli	Backend Systems Engineer	FastAPI async architecture; REST API design; Locust load testing (100+ RPS, <80
Sai Dheeraj Gollu	Data Pipeline Engineer	ETL pipelines for 100K+ recipes; SpaCy NER filtering (87% precision); FAISS embed

Table 14 — Team member contributions

12. Conclusion

PlatePlanner demonstrates that a carefully composed polyglot architecture — combining relational, graph, vector, and key-value storage — can power a user-facing AI application that is simultaneously *fast* (sub-80ms end-to-end), *accurate* (92–95% on key metrics), and *memory-efficient* (90 MB FAISS index covering 2.2M recipes). The hybrid semantic + discrete ranking algorithm and context-aware Neo4j substitution graph are the two key technical contributions beyond off-the-shelf components.

Future directions include fine-tuning the recipe encoder on user interaction data, adding real-time inventory tracking via barcode scan, and expanding the substitution graph with crowd-sourced feedback loops.

PlatePlanner

San Jose State University · CMPE 295B · April 2026

Sandilya Chimalamarri · Sai Priyanka Bonkuri · Pavan Charith Devarapalli · Sai Dheeraj Gollu